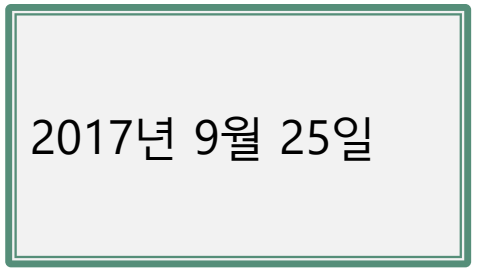
객체지향 프로그래밍 고급

7강 : 상속

[복습] 복사생성자

오늘 날짜를 의미하는 세 정수를 입력받은 후 아래와 같이 출력되는 Date 클래스를 정의하고 복사생성자를 정의하시오.



Chapter 7

클래스 상속

protected 선언

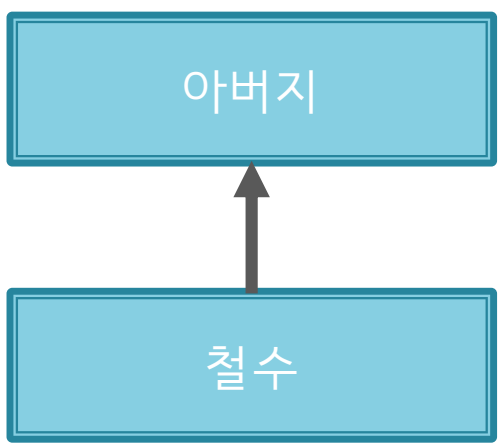
상속의 개념

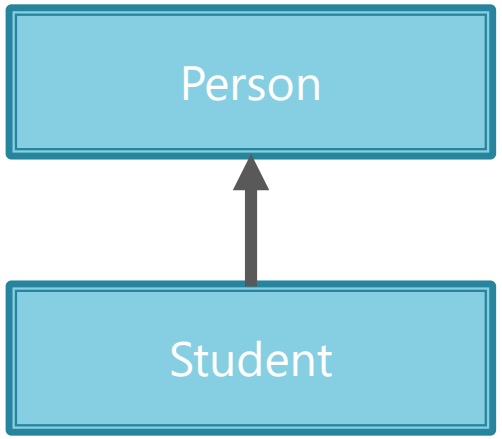
❖ 상속의 예1

철수는 아버지로부터 좋은 목소리와 큰 키를 물려받았다.

❖ 상속의 예2

Student클래스가 Person 클래스를 상속한다.





상속 클래스

❖상속(inheritance)

- 한 클래스가 다른 클래스에서 정의된 속성을 이어받아 그대로 사용
- 이미 정의된 클래스를 바탕으로 필요한 기능을 새로 추가하여 정의
- 소프트웨어 재사용

❖ 기본클래스(base class), 부모클래스(parent class), 슈퍼클래스

- 이미 만들어진 기존의 클래스

❖ 파생클래스(derived class), 자식클래스, 하위클래스

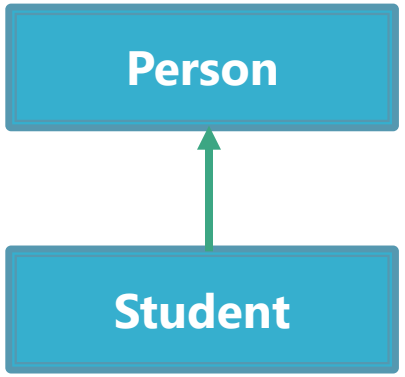
- 상속받아 새로 만들어지는 클래스

상속 클래스

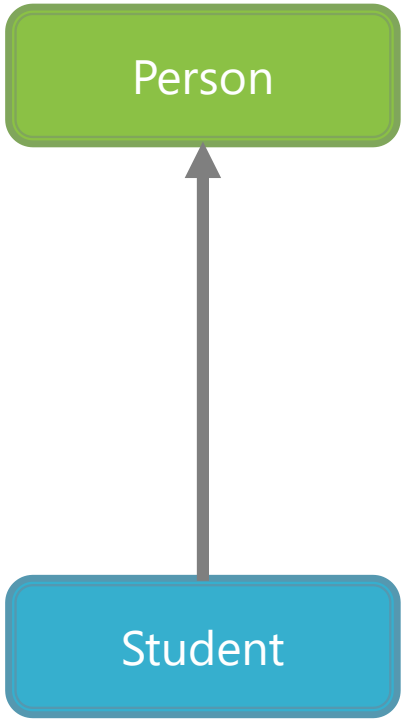
class 파생클래스명 : public[private]기본클래스명
{
}

```
class Person
{
. . .
};

class Student : public Person
{
. . .
};
```



상속의 방법



- 이름
- 나이

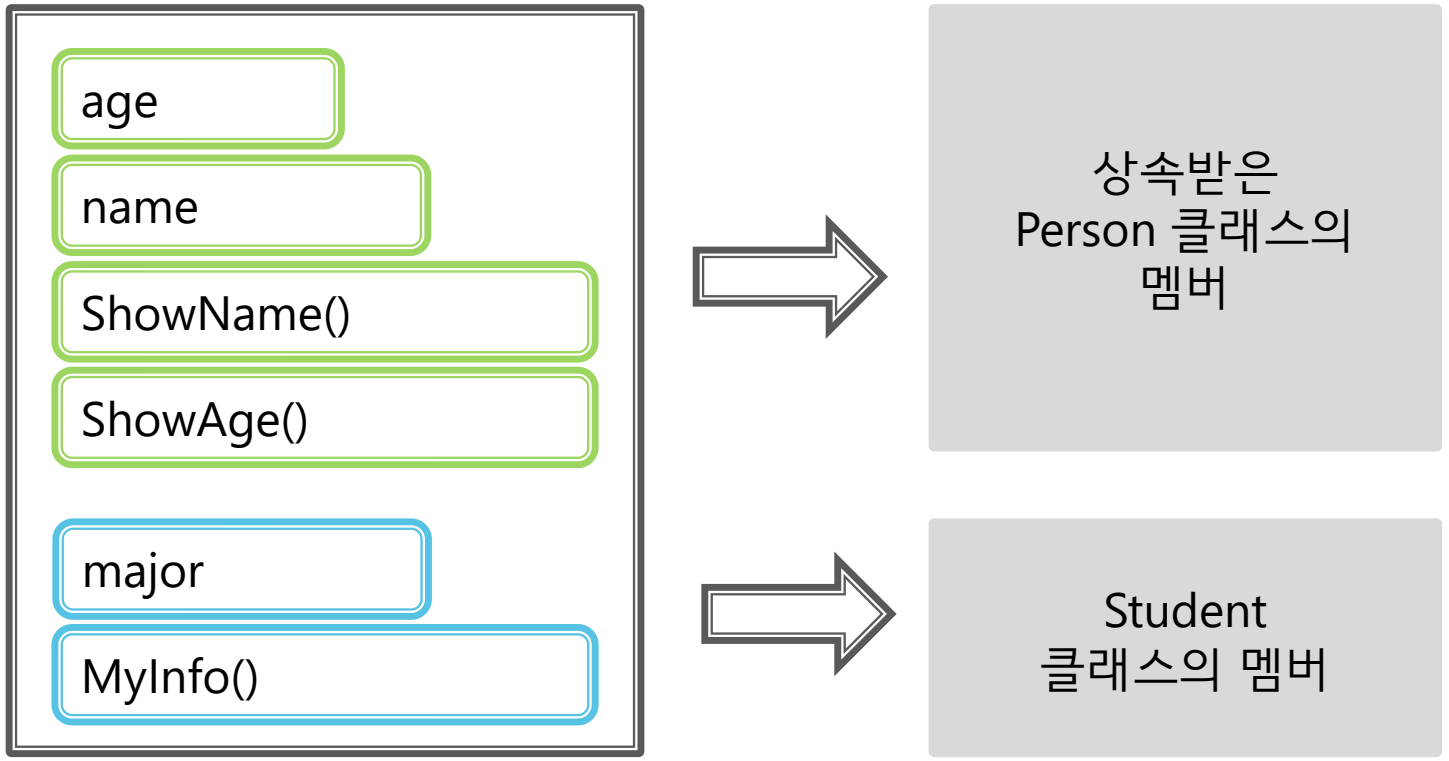
- 전공

```
class Person
멤버변수
  string name;
  int age;
멤버함수
  void ShowName();
  void ShowAge();
```

```
class Student
멤버변수
  string major;
멤버함수
  void MyInfo();
```

상속의 방법

- 자식클래스인 Student가 갖는 멤버변수와 멤버함수



상속의 방법

```
class Person
{
private:
	int age;
	string name;
public:
	Person(int myage, string myname) : age(myage)
	{
		name = myname;
	}
	void ShowName() const {
		cout << "name : " << name << endl;
	}
	void ShowAge() const {
		cout << "age : " << age << endl;
	}
};
```

```
class Student : public Person
{
private:
	string major;
public:
	Student(string myname, int myage, string mymajor)
		: Person(myage, myname)
	{
		major = mymajor;
	}
	void MyInfo() const {
		ShowName();
		ShowAge();
		cout << "major : " << major << endl;
	}
};
```

상속의 방법

```
int main()
{
	Student std1("Hong Gildong", 22, "Game");
	std1.MyInfo();

	Student std2("Jang Donggun", 24, "Computer Science");
	std2.MyInfo();
	return 0;
}
```

상속할 때 주의할 점(1)

① 부모클래스의 생성자는 어디에서 불리는가?

Student 클래스의 생성자가 Person 클래스의 멤버까지 초기화해야 하는가?

Student 클래스는 Person 클래스의 멤버까지 초기화해야 한다.

왜냐하면, Student 객체가 생성되면 그 객체 안에 다음의 멤버 변수가 존재하기 때문이다.

- Person의 멤버변수 age, name
- Student의 멤버변수 major

Student 클래스의 생성자가 Person 클래스의 멤버를 어떻게 초기화는 것이 좋은가?

Student 클래스의 생성자는 Person 클래스의 생성자를 호출해서 Person 클래스의 멤버를 초기화하는 것이 좋다.

➲ 자식클래스의 생성자에서 불린다.

상속할 때 주의할 점(1)

```
Student(string myname, int myage, string mymajor) : Person(myage, myname)
{
    major = mymajor;
}
```

상속할 때 주의할 점(2)

②자식클래스의 멤버함수에서 부모클래스의 멤버변수(private)에 접근할 수 있는가?

Student 클래스의 멤버함수 내에서 Person클래스에 private으로 선언된 멤버변수(age, name)에 접근이 가능한가?

- 접근제한의 기준은 클래스이다.
- 클래스 외부에서는 private 멤버에 접근이 불가능하다.
- 따라서 Student의 멤버함수 내에서는 Person의 멤버변수에 직접 접근이 불가능하다.

➲ 접근할 수 없다.

파생클래스의 객체 생성

```
class Parent
{
private:
	int pNum;
public:
	Parent() : pNum(20)
	{
		cout << "Parent()" << endl;
	}
	Parent(int n) : pNum(n)
	{
		cout << "Parent(int n)" << endl;
	}
	void ShowParentData()
	{
		cout << "pNum : " << pNum
	<< endl;
	}
};
```

```
class Child : public Parent
{
private:
	int cNum;
public:
	Child() : cNum(30) {
		cout << "ChildI()" << endl;
	}
	Child(int n) :cNum(n) {
		cout << "Child(int n)" << endl;
	}
	Child(int n1, int n2) : Parent(n1), cNum(n2)
	{
		cout << "Child(int n1, int n2)" << endl;
	}
	void ShowChildData()
	{
		ShowParentData();
		cout << "cNum : " << cNum << endl;
	}
};
```

파생클래스의 객체 생성

```
int main()
{
	cout << "case1 _________" << endl;
	Child ch1;
	ch1.ShowChildData();
	cout << "_________________" << endl;

	cout << "case2 _________" << endl;
	Child ch2(12);
	ch2.ShowChildData();
	cout << "_________________" << endl;

	cout << "case3 _________" << endl;
	Child ch3(23, 26);
	ch3.ShowChildData();
	cout << "_________________" << endl;

	return 0;
}
```

파생클래스의 객체 생성 과정

- ❖ 파생 클래스의 객체 생성 과정에서 기본 클래스의 생성자는 100% 호출된다.
- ❖ 파생 클래스의 생성자에서 기본클래스의 생성자 호출을 명시하지 않으면, 기본 클래스의 void 생성자가 호출된다.

파생 클래스 객체의 소멸과정

파생클래스의 객체가 소멸될때는, 파생클래스의 소멸자가 실행되고 난 후, 기본클래스의 소멸자가 실행된다.

```
~Parent()
{
    cout << "~Parent()" << endl;
}


~Child()
{
    cout << "~Child()" << endl;
}
```

[예제1]

Person클래스는 이름(char* name)과 학번(char* stuId)을 멤버변수로 갖는 클래스이다.
Student클래스는 Person클래스의 파생 클래스이고 전공(char* major)을 멤버변수로 갖는다.
Person클래스와 Student클래스의 멤버변수를 모두 동적할당하고, 아래 데이터를 출력하시오.

```
int main()
{
	Student std1( " 장동건",  " 수학과");
	std1.ShowMyInfo();
	Student std2( " 아이유",  " 게임학과");
	std2.ShowMyInfo();
	return 0;
}
```

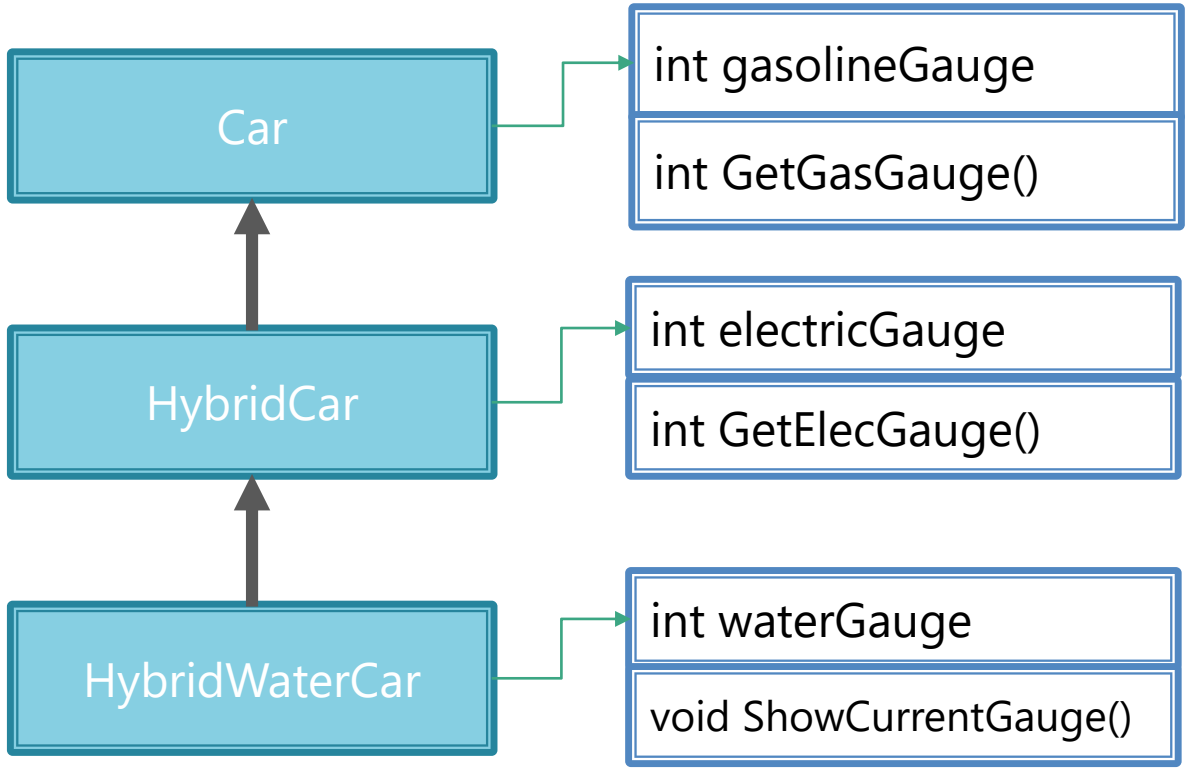
[예제2]

이름과 나이를 멤버변수로 갖는 MyFriendInfo 클래스가 있다. MyFriendDetailInfo 클래스는 MyFriendInfo 클래스로부터 상속받고 주소와 핸드폰번호를 멤버변수로 갖는다.
이 때 이름은 char* 타입이며 동적할당한다.
아래 데이터를 갖는 클래스 객체를 생성하고 출력하라.

```
("김국진", 24, "경기도이천", "010-3423-3330");
("이수경", 22, "서울 서초구", "010-3539-9458");
```

[예제3] 다음 클래스를 구현하시오.

Car, HybridCar, HybridWaterCar 클래스의 상속 관계는 아래와 같다. 아래의 출력 결과가 나오도록 프로그래밍하라.




```
int main()
{
	HybridWaterCar hbCar(79, 65, 34);
	hbCar.ShowCurrentGauge();
	return 0;
}
```

출력결과:

```
잔여가솔린:79
잔여전기량:65
잔여워터량:34
```

Chapter 6

클래스 상속

protected 선언

세가지 형태의 상속

```
class Derived : public Base
{
}

class Derived : protected Base
{
}

Class Derived : private Base
{
}
```

세가지 형태의 상속

❖ **접근제어 지시자가 허용하는 접근의 범위**
private < protected < public

❖ **protected로 선언된 멤버변수는 이를 상속하는 파생클래스에서 접근이 가능하다.**

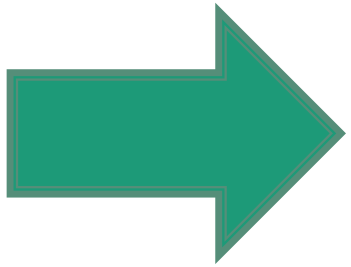
세가지 형태의 상속

```
class Parent
{
private:
    int num1;
protected:
    int num2;
public:
    int num3;
    void ShowData()
    {
      cout << num1 << ", " << num2 << ", "
              << num3 <, endl;;
    }
};
```

```
class Child : public Parent
{
public:
    void ShowParentMember()
    {
      cout << num1;
      cout << num2;
      cout << num3;
    }
};
```

protected 상속

```
class Parent
{
private:
    int num1;
protected:
    int num2;
public:
    int num3;
};
```



```
class Child : protected Parent
{
접근불가:
    int num1;
protected:
    int num2;
protected:
    int num3;
};
```

```
class Child : protected Parent
{

};
```

protected 상속

```
class Parent
{
private:
    int num1;
protected:
    int num2;
public:
    int num3;
    Parent() : num1(1), num2(2), num3(3)
{ }
};
```

```
class Child : protected Parent
{
};

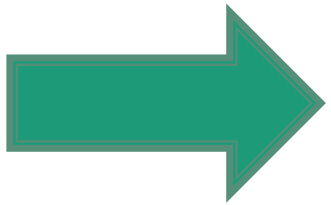
int main()
{
    Child drv;
    cout << drv.num3 << endl;
    return 0;
}
```

➲**컴파일 결과는?**

Child의 멤버변수 drv는 상속된 클래스에서만 protected로 변경되어 사용가능
main()함수에서는 사용할 수가 없다.

private 상속

```
class Base
{
private:
    int num1;
protected:
    int num2;
public:
    int num3;
};
```



```
class Derived : private Base
{
접근불가:
    int num1;
private:
    int num2;
private:
    int num3;
};
```

```
class Derived : private Base
{

};
```

➲**컴파일 결과는?**

num2와 num3는 derived 클래스내에서만 접근이 가능한 멤버가 된다.

public 상속

private을 제외한 나머지는 그냥 그대로 상속한다.

[예제5]

책을 의미하는 Book 클래스와 전자책을 의미하는 EBook클래스를 정의하고자 한다. 다음의 형태로 클래스의 상속관계를 구성하고, 아래와 같이 출력되도록 클래스를 완성하시오.

```
class Book
{
private:
    char *title;
    char *isbn;
    int price;
    . . .
};
class Ebook : public Book
{
private:
    char *DRMKey;
    . . .
};
```

```
int main()
{
    Book book("C++ 프로그램", "555-12345-890-0", 20000");
    book.ShowBookInfo();
    cout << endl;
    Ebook ebook("C++ 프로그램 ebook", "555-12345-890-1", 10000", fdx98dfx);
    ebook.ShowBookInfo();
    return 0;
}
```

```
제목: C++ 프로그램
ISBN: 555-12345-890-0
가격:20000

제목: C++ 프로그램 ebook
ISBN: 555-12345-890-1
가격:10000
인증키: fdx98dfx
```

[예제6]

가로, 세로의 길이를 갖는 사각형 Square를 기본 클래스로 정의한 후, 이를 상속받아 가로, 세로, 높이를 갖는 직육면체 Cube 클래스를 정의하시오.
아래의 사각형 면적과 직육면체의 부피를 구하시오.

사각형 (5, 10)
직육면체 (5, 10, 3)

[예제7]

Point 클래스는 x, y 좌표를 표현하는 클래스이다.
Rectangle 클래스는 Point의 파생클래스로서 사각형을 표현한다.
사각형의 면적을 구하라.

```
좌표 : (0,0), (10,10)
사각형의 면적 : 100
```

[예제9]

아래 BaseArray 클래스를 상속받아 큐처럼 동작하는 MyQueue 클래스를 구현하라.

```
class BaseArray {
private:
    int capacity;
    int *mem;
protected:
    BaseArray(int capacity=100) {
        this->capacity = capacity;
        mem = new int [capacity]; }
    ~BaseArray() { delete [] mem; }
    void put(int index, int val) {mem[index] = val };
    int get(int index) { return mem[index]; }
    int getCapacity() { return capacity; }
};
```

```
int main() {
    MyQueue mq(100);
    int n;
    cout << "큐에 다섯 개의 정수를 입력하라. " ;
    for (int i=0;i<5;i++) {
        cin >> n;
        mq.inqueue(n);
    }
    cout << "큐의 용량: " << mq.capacity() << endl;
    cout << "현재 큐의 크기: " << mq.length() <<endl;
    while (mq.length() != 0) {
        cout << cout << mq.dequeue() << ' ';
    }
    cout << endl << "큐의 현재 크기: " <<mq.length();
```

상속의 조건

IS-A 관계

NoteBook IS-A Computer

HAS-A 관계

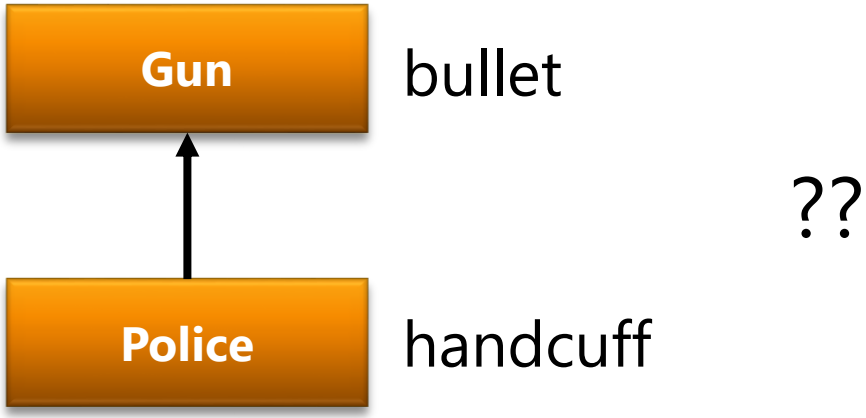
Police HAS-A Gun

[예제8] 다음 조건을 만족하는 클래스를 구현하라.

모든 경찰은 총과 수갑을 소지한다. 경찰이 갖는 총알의 개수와 수갑의 개수는 저마다 다르다.

경찰이 범인에게 총을 쏘거나 수갑을 채우는 것을 프로그램으로 구현할 때 클래스의 상속 관계는 어떻게 표현하면 좋을까?

총은 총알을 소유하고 있고 경찰은 총과 수갑을 소유하고 있다.



[과제3]

5개의 총알, 2개의 수갑을 소유한 경찰과 권총이 없고 1개의 수갑을 소유한 경찰이 있다. 경찰들이 총을 쏘고 수갑을 채우는 것을 표현하는 프로그램을 구현하라. 이 때 class Gun과 class Police를 상속이 아닌 방법으로 정의하시오.

```
int main()
{
	Police pman(5, 2);
	pman.Shot();
	pman.PutHandcuff();
	pman.ShowHandcuff();

	Police pman2(0, 1);
	pman2.Shot();
	pman2.PutHandcuff();
	pman2.ShowHandcuff();

	return 0;
}
```

```
class Gun
{
private:
	int bullet;  //장전된 총알의 수
public:
	Gun(int bnum) : bullet(bnum) { }
	void Shot() { }
	void ShowBullet() { }
};
```

```
class Police
{
private:
	int handcuffs;
	Gun *pistol;
public:
	Police(int bnum, int bcuff) : handcuffs(bcuff)
	{
		if (bnum > 0)
			pistol = new Gun(bnum);
		else
			pistol = NULL;
	}
```